INTEGRATION GUIDE
SDK GETNET

VERSION 1.18

Sumário

1 Introduction

This document describes the use of this package, containing the SDK and documentation, as well as providing an overview of the SDK and its security functions.

This document also contains the conventions for secure implementation and the flow for UI callback integration, authentication, payment, and configurations.

The Getnet SDK aims to provide a simple and secure interface for executing contactless payment transactions, without the integrator having to worry about detailed security configurations and handling sensitive data, such as device attestation and card data processing.

2 Overview

The Getnet SDK provides contactless payment operations using a card (or digital wallet device) without the need to insert the card.

The available operations are credit payment with or without installments, debit payment and sale cancellation.

For these operations, the integrator does not need to worry about manipulating sensitive data, as our SDK only requires the transaction start data, such as the value.

The SDK was developed for the Android platform with access to an NFC reader on the device.

The figure below shows an overview of the SDK components and communication boundaries.

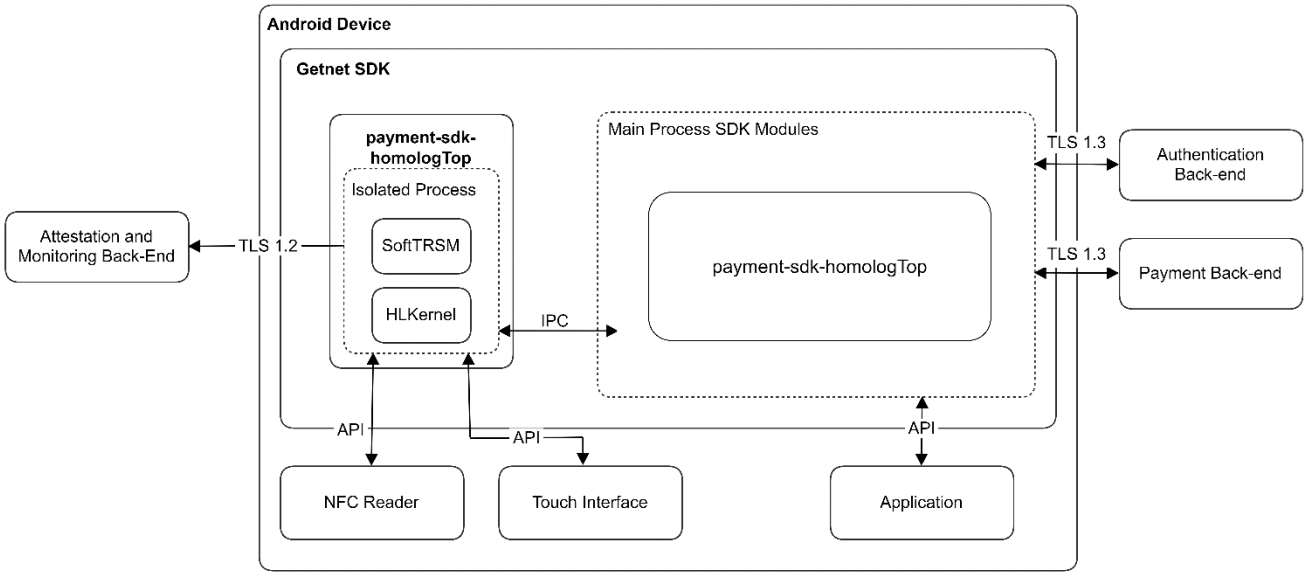


Figure 1. Component Overview

2.1 Modules

Our SDK is made up of two essential modules, seen in Figure 1, for integration, they are:

- **payment-sdk-homologTop:** Main module, performs authentication functions and initiates transaction process.
- **sdk-nfc-reader:** Auxiliary library for NFC support (only need to be present on the available maven repository).

2.2 Memory isolation

As seen in figure 1, all operations related to Android NFC card reading input, communication with the Android touch interface (such as PIN input) and security attestation take place within an isolated process with separate memory. other elements.

This isolated process, managed by the HLKernel and SoftTRSM submodules, is also responsible for ensuring the security of sensitive transaction information, such as card and PIN data, through strong encryption, allowing the rest of the SDK to transport this data safely after receiving them from inter process communication (IPC).

2.3 Encryption

As stated in the chapter on memory isolation, all sensitive transaction data is handled within an isolated process, which means that the integrator does not need to worry about encryption keys, secure communication, or any other security setting for the operation of transaction.

It is important that the SDK is kept up to date with the updates provided (chapter 4) so that the application is protected from possible new vulnerabilities. These updates will also implement new features and security fixes as the operating system makes them available/requires them.

2.4 Data Management

To be able to fulfill a complete transaction, the SDK needs to handle some assets that are described in Tables 1 and 2.

To capture the card data, the SDK solution needs a set of parameters that will configure its behavior or take part in the communication with the card. All these data are inherently non-sensitive, being received in plain format.

Data	Description
Amount, Authorized	Transaction amount
Amount, Other	Amount for cancellation
Date	Current date
Time	Current time
AID List	List of applications to be looked for in the presented card, containing version number, terminal capabilities, terminal type, action codes, default kernel index, contactless floor limit, contactless transaction limit, CVM required limit, TDOL, and DDOL
CNPJ	Used for authentication and merchant identification.

Table 1. Remote Process Data

On the other hand, the Main Process (i.e., the user mode application) receives a set of data captured from the card or generated by the Remote Process (i.e., the secure, isolated HLKernel solution space).

The criteria for determining an asset in application scope as critical are its purpose, intended usage, and confidentiality, integrity, and resilience requirements. Card data elements (e.g. PAN sequence, PIN, and card's expiration date) or the crucial component in secure procedures that handle sensitive data (e.g. cryptographic and encoding functions), are defined as critical assets.

Confidentiality. All the sensitive data listed here is confidential. Revealing their contents outside the scope where they were meant to be processed (i.e., the HL Kernel SDK and the Soft TRSM gateway) may lead to financial losses to their legitimate owners, as well as to the entities that were supposed to protect it.

Integrity. Operations done over critical assets must ensure their integrity, i.e., that they were not tampered with, changed, added, or removed from their original context, producing or with the possibility to produce a result that was not intended by its legitimate owner.

Resilience. Even under an adverse scenario, the items defined as critical assets keep its confidentiality and integrity due to the multiple security mechanisms implemented to protect them. These mechanisms include, but are not limited to, root detection, dynamic instrumentation detection, replay detection, and application tampering detection.

Name	Operations	Location	Security
Card's PAN Type: Account Data Description: Card's PAN content	The PAN (personal account number) is read from the card and stored in the transaction's tag pool until it is completed. Before the contactless read process scope is left, the PAN is encrypted using a transaction key, a masked version is created (leaving visible the first six and the last four digits) and the original number is cleared.	Device volatile memory.	**[Confidential]** The plain PAN exists between its read from the card until it is encrypted using the transaction key. It is never stored in the device, nor leaves the scope of its class.
Card's track 2 equivalent data Type: account Data Description: card's track 2 content	The track 2 equivalent data is read from the card and stored in the transaction's tag pool until it is completed. Before the contactless read process scope is left, the track 2 equivalent data is encrypted using a transaction key and the original, plain content, is cleared	Device volatile memory.	**[Confidential]** The plain track2 equivalent data exists between its read from the card until it is encrypted using the transaction key. It is never stored in the device, nor leaves the scope of its class
Card's expire date Type: account Data Description: card's expire date content	The expire date is read from the card and stored in the transaction's tag pool until it is completed. Before the contactless read process scope is left, the expire date is	Device volatile memory.	**[Confidential]** The plain expire data exists between its read from the card until it is encrypted using the transaction key. It is never

	encrypted using a transaction key and the original, plain content, is cleared		stored in the device, nor leaves the scope of its class
DRBG's seed Type: DRBG component Description: seed fed to the DRGB component	Before a payment transaction can start, a new seed for the DRBG is created. After feeding it to the DRBG, the seed is cleared. The DRBG is implemented according to the SP 800-90A It is generated from different entropy sources.	Device volatile memory.	**[Confidential]** The seed is needed to feed the application's DRBG. Random numbers generated with it is used by the contactless classes as the EMV's Unpredictable Number, as well as to create the device's ECDHE private key.
ECDHE private key Type: cryptography component Description: generated key with 521 bits length	An ECC private key is generated and kept in device's volatile memory. The corresponding public key is calculated and sent to the backend, where a shared secret is derived to encrypt the transaction keys	Device volatile memory.	**[Confidential]** The private key is generated just before the remote attestation message is sent to the backend and exists until a response is received (or the connection is lost).
Key transport key Type: cryptography component Description: generated key using X25519/P521 elliptic curve	Using the ECC private key and the backend's calculated ECC public key, a shared key is derived and used a transport key to decrypt the transaction keys	Device volatile memory.	**[Confidential]** The KTK is derived when the key response message is received and is calculated based on the device's EDCHE private key and the backend's EDCHE public key. It will be erased as soon as the transaction keys cryptogram is opened.
Sensitive data encryption key Type: cryptography component Description: AES algorithm generated key	Ephemeral sensitive data keys received from the backend are used to encrypt the PAN, track 2 equivalent data and card expire date. They are kept only in device's volatile memory and exists between the start of a payment transaction and the data encryption is completed, or the transaction is aborted by any reason, whichever happen first	Device volatile memory.	**[Confidential]** It encrypts the PAN, track 2 equivalent data, and expire date, as well as the whole payment request message
PIN encryption key Type: cryptography	Ephemeral keys received from the backend used to encrypt	Device volatile memory.	**[Confidential]** Encrypts the PIN

component Description: AES algorithm generated key, CBC mode as per ISO 9564-1: 2017 Format 4	the cardholder's PIN. They are kept only in device's volatile memory and exists between the start of a payment transaction and the key encryption finishing, or up until the transaction is aborted by any reason, whichever happen first		
PIN Type: authentication credentials Description: cardholder's secret code	The cardholder's PIN is captured whenever the selected CV method is "online PIN". It is immediately encrypted using the PIN encryption key and erased. The PIN's cryptogram is stored in the transaction's context	Device volatile memory.	**[Confidential]** Once entered, the PIN is immediately encrypted using the PIN encryption key and erased. The PIN's cryptogram is stored in the transaction's context and will be added to the payment authorization request
Device attestation data Type: attestation Description: data collected by the COTS device for attestation purposes	Data collected by the device to ensure the integrity of its runtime environment	Device volatile memory.	**[Confidential]** Ephemeral, encrypted by JWE public key before being sent to the backend for evaluation
Device attestation result Type: attestation Description: flag indicating the success or failure in attesting the COTS device	Result received from the backend, telling whether the attestation was successful or not	Device volatile memory.	**[Integrity]** Signature checked by JWS public key
COTS device authentication certificate Type: device authenticity Description: verifies the authenticity of attestation requests	Sent along with the attestation requests to allow the backend verifies its authenticity	Device persistent storage.	**[Integrity]** Signature by the backend's COTS device authentication certificate authority private key
COTS device authentication private key Type: device authenticity Description: ensures attestation requests	Signs attestation requests before sending them to the backend	Device key store.	**[Confidential]** Key is not directly accessible by the software, being held in the device's key store for its entire lifetime

authenticity			
Hash of COTS device authentication CSR Type: device authenticity Description: hash calculated over COTS device authentication CSR	Hash is included in the Play Integrity payload and verified by the backend before signing the COTS device authentication CSR	COTS' volatile memory	**[Integrity]** Included in the Play Integrity payload

Table 2. Sensitive Assets

The only data that the integrator is responsible for is the Amount (Table 1), its usage is described at the transaction chapter, all other data is managed by the SDK and all sensible data is encrypted and communicated through secure TLS channels with strong cipher suite.

2.5 Attestation and Monitoring

Our SDK has an Attestation and Monitoring system, managed by SoftTRSM, which guarantees the integrity of the device and the application where transactions will be executed, carrying out checks using RASP, Root Detection, and other security validations.

There is no need for the integrator to worry about these implementations, all configuration is done internally by our SDK and backend systems.

Table 3 below show the SDK behavior applied in case each of these tampers are detected.

Tamper	**Behavior**
Genuine Installation	Device blocked
Google Integrity API	Device blocked
Root Detection	Device blocked
Malware Detection	Device blocked
Developer or Accessibility Options	Transaction blocked
Camera Detection	Transaction blocked
Dexguard tamper detection	Device blocked
Checksum validation failed	Device blocked
Entropy sensors usage failed (microphone, accelerometer)	Transaction blocked

Table 3. Tamper behavior

3 Integration

Integration of the GetNet SDK comes down to two major steps:

1. **Authenticating to the SDK:** stage in which it will be necessary to integrate the partner's APIs with the GetNet APIs, aiming to enable the terminal to perform payments.
2. **Integrating to the SDK:** integration of the SDK library with the partner App.

3.1 Partner Authentication

The authentication step is what allows the partner's application to perform payment transactions through GetNet systems. This step starts in the partner's backend, holder of credentials, informing them through its application to the GetNet SDK. In turn, the SDK has the role of communicating with the GetNet Backend, enabling or not the terminal.

The development required on the partner backend is described in the document “TAP On Phone Silent Auth-1.0.pdf”.

This integration will result in the provisioning of two tokens (access token and app token) that are used for the application to be authenticated in the SDK via the API described in chapter 8.

4 Requirements

This chapter describes some needs of the SDK library for its correct functioning and to ensure secure integration.

4.1 Android Version

Currently the Getnet SDK supports Android versions 12 to 14. This is constantly updated to keep the security up to date according to security updates released by Android.

4.2 Permissions

The RECORD_AUDIO permission is mandatory to ensure strong keys for transaction encryption by collecting device microphone entropy. Do Not Disturb permissions are required for PIN capture screen control.

Locations permissions are required for Getnet's anti-fraud system.

These permissions must be requested in runtime by the application before any transaction is processed.

For the SDK to work correctly, the following instructions are present in the SDK's Android manifest and will be merge into the application's manifest:

```
<uses-permission android:name="android.permission.INTERNET" />
<uses-permission
android:name="android.permission.ACCESS_NETWORK_STATE" />
<uses-permission android:name="android.permission.SYSTEM_ALERT_WINDOW"
/>
<uses-permission android:name="android.permission.VIBRATE" />
<uses-permission
android:name="android.permission.ACCESS_NOTIFICATION_POLICY" />
<uses-permission android:name="android.permission.NFC" />
<uses-permission
android:name="android.permission.RECEIVE_BOOT_COMPLETED" />
<uses-permission android:name="android.permission.RECORD_AUDIO" />
<uses-permission
android:name="android.permission.ACCESS_COARSE_LOCATION" />
    <uses-permission
android:name="android.permission.ACCESS_FINE_LOCATION" />
<uses-permission
android:name="android.permission.HIDE_OVERLAY_WINDOWS" />
    <uses-permission
        android:name="android.permission.QUERY_ALL_PACKAGES"
        tools:ignore="QueryAllPackagesPermission" />
```

4.3 Android Manifest

In addition to permissions, through your application's AndroidManifest.xml, you must ensure that application backup and resizable activities are both disabled, as the former allows data to be kept on the device after the application has been disabled (uninstalled) , and the latter allows multiple applications to be opened at the same time, which imposes a security risk when running an application in parallel with sensitive operations. This is accomplished by adding the following attributes to the application tag:

```
<application
    android:allowBackup="false"
    android:dataExtractionRules="@xml/data_extraction_rules"
    android:fullBackupContent="false"
    android:resizeableActivity="false"
/>
```

The DataExtractionRules is needed for Android 12 or higher to specify which data is allowed to be preserved. Below is the configuration necessary to avoid keeping any data from a deleted application, not applying this configuration will result on non-compliance to the PCI MPoC:

```
<!-- Rules required for disallowing Android Auto Backup in Android
12 or higher -->
<data-extraction-rules>
    <cloud-backup>
        <exclude domain="root" />
        <exclude domain="file" />
        <exclude domain="database" />
        <exclude domain="sharedpref" />
        <exclude domain="external" />
    </cloud-backup>
    <device-transfer>
        <exclude domain="root" />
        <exclude domain="file" />
        <exclude domain="database" />
        <exclude domain="sharedpref" />
        <exclude domain="external" />
    </device-transfer>
</data-extraction-rules>
```

4.4 Obfuscation

Libraries are obfuscated with obfuscation provided by Dexguard to guarantee the SDK code integrity preventing external modification, some methods and classes have retained their

signatures to allow the SDK implementation, but for another layer of protection the integrator application can obfuscate all classes and methods of the libraries and the app code itself before releasing the application to the store.

Many third-party obfuscators have a string encryption feature where all strings declared in the code are encrypted, so that all parameters passed to the libraries are more secure by being defined directly in the code instead of being loaded from assets or other external resources. (gateway host address, certificates, etc.).

4.5 Updates and Newsletters

It is important that the integrating application is always using the latest available version of the Getnet SDK to guarantee transaction security.

New versions of the SDK are constantly published on our FTP server, to which access is provided after purchasing this product, when new features are available or when security fixes/vulnerabilities are implemented.

Release notes for each version will be made available within the SDK package along with these and other integration documents.

Be sure to continually check the inbox of your registered email address (through our onboarding process) as we send new version updates and security guidance on potential vulnerabilities.

Communications about security vulnerabilities are made through the same channel. When a vulnerability is identified by our team, communication will be sent at a maximum period of 7 days, either with a security patch or a mitigation alternative when the patch is expected to take more than 7 days to be applied.

4.6 Kotlin and Java version

The minimum Java version supported for the SDK to work correctly is 17. As for Kotlin the minimum supported is 2.0.21.

5 API Documentation

The SDK integration package contains SDK API documentation in HTML format, describing all available methods and their use.

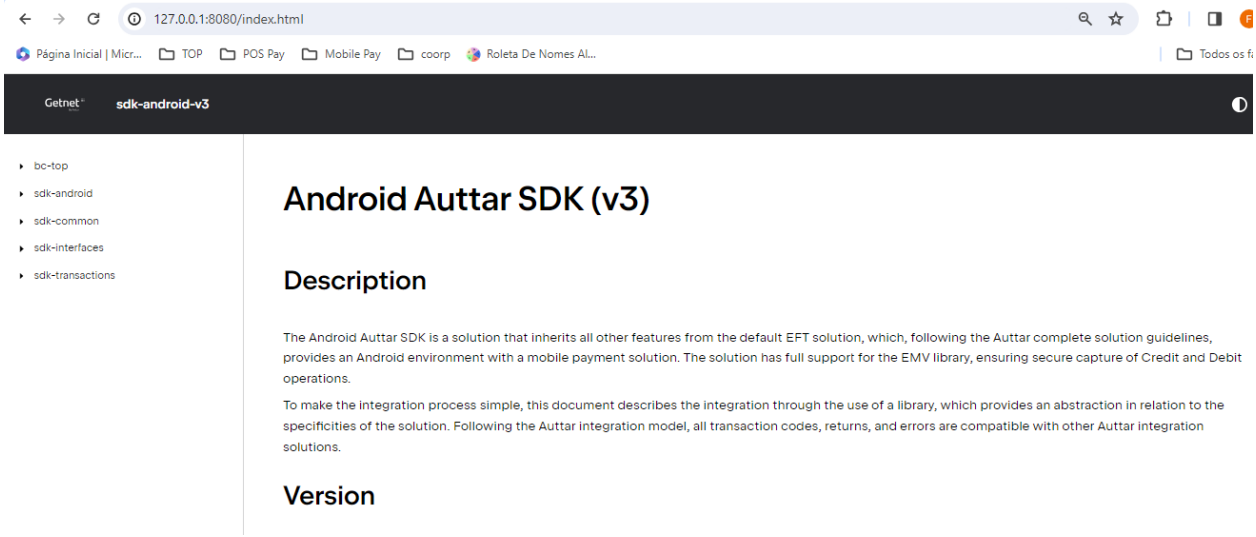
This document is outdated, but still useful for checking classes and enums descriptions.

The SDK API describes methods and classes beyond those contained in this document for a specific use. The “documentation” folder has the documentation and through it is possible to use it minimally by opening index.html, however, to view it completely it is necessary to have the NPM application installed and execute the following commands in the documentation directory:

```
npm i
npm run start
```

Once the above commands have been executed, a small web server will be started, which must be accessed through the browser at http://127.0.0.1:8080/. The hyperlinks in this document are accessible through this URL.

Our API page will be accessible at this address. Like the page seen below:



Browsing this page, you will find the modules and packages present in the SDK and the signature on the methods available for integration, as well as details of the parameters that must be used.

As a guide for this implementation, we will describe the steps for integration in the next chapters of this document, from UI integration to carrying out a transaction.

6 Library Integration

The Getnet SDK is compatible with Android Studio and is distributed through a Maven repository that should be referenced in the project. The provided zip file containing the SDK must be unpacked and referenced in the project through the gradle.build file, as shown in the example below.

```
  dependencies {
    repositories {
        maven {
            name = "auttar"
            url = "file://C:\\Diretorio\\SDK\\Auttar\\"
        }
    }


    implementation 'br.com.auttar.mobile: payment-sdk-top-
release:X.Y.Z'
}
```

7 Initialization

Before proceeding to the next steps make sure your Application Class extends the TopApplication from the SDK and on the onCreate method of the Application make sure to initialize the SDK through the init method from the Auttar object passing the extended TopApplication class.

The extended application class from the integrating App can be extended further and there's no problem being an open class if it is required by the app's architecture, as long as the Auttar.initHomolog(), Auttar.initProduction() or Auttar.initStandalone() method is invoked in the onAppCreated() method.

Exemple of a class extending TOPApplication:

```
  class Application : TOPApplication() {
    override fun onAppCreated() {
        super.onAppCreated()
        Auttar.initHomolog(this)
    }
}
```

The SDK facades are obtained through the same Auttar object. This object (IAuttarSDK) provides the getSDK and getConfigSDK (details on chapter 11) methods.

Through the getSDK method it is possible to obtain the main facade, responsible for enabling authentication and starting the execution of transactions.

Below is an example to obtain the facade:

```
  val auttarSDK: IAuttarSDK = Auttar.getSDK()
```

8 Interface and Callback Integration

Callbacks are part of the way the SDK works, being the way the library interacts with the integrating application. These callbacks are triggered after the start of a transaction (chapter 9) to indicate the need for interaction with the user, a process step information and message suggestions for the user.

With these callbacks, the integrating application can customize the user interface as desired during the transaction, except for the card reading keyboard, for security reasons this keyboard is managed internally by the Getnet SDK.

IMPORTANT: The same transaction amount passed to the payment API must be displayed to the user during the transaction flow. Not following this requirement will result in non-compliance to PCI MPoC.

The implementation of callbacks is done through the ClientUserInterface present in the sdk-interfaces module. Below is a simplified implementation of the interface. For more details on the parameters and functions you can check the API documentation of this interface.

```
  private val _clientUserInterface = object : ClientUserInterface {
    override fun askGenericCapture(
        display: String, mask: String, maxLength: Int, editionMode: Int
    ) {    /*App implementation*/       }

    override fun askForValue(display: String) {/*App implementation*/}

    override fun messageDisplay(message: String) {/*App implementation*/}

    override fun showContactlessCard(status: String) {/*App implementation*/
}

    override fun askForDate(display: String, mask: String) {/*App
implementation*/}

    override fun cleanDisplay() {/*App implementation*/}

    override fun askForConfirm(display: String) {/*App implementation*/}

    override fun askSelect(display: String, options: Array<String>) {/*App
implementation*/}

    override fun showInsertCardMessage(display: String) {/*App
implementation*/}

    override fun showPin(display: String) {/*App implementation*/}


}
```

8.1 ClientUserInterface methods

The ClientUserInterface methods normally require an action from the App. Some of the actions mentioned require a direct response to the SDK, such as date capture for the cancellation transaction.

These responses are sent asynchronously through the ResponseUserInterface interface. To obtain this interface, simply access it through the *getObjectResponseCallback* method of the IAuttarSDK main facade (chapter 7).

If the action is canceled by the user, it is important that the interface's cancel method is called so that the operation ends.

Below we have descriptions of each method, its description and what action should be taken in response.

askGenericCapture
Description: Indicates the need to capture a generic String, a mask and string length indications are sent, an example is the capture of CPF or telephone number.
Action: A field for string capture, meeting the requirements received by parameters, must be provided and the captured string must be sent through the *askForGeneric* method of the ResponseUserInterface.

askForValue
Description: Indicates the need to capture a string with a monetary value.
Action: A field for string capture must be provided and the captured string must be sent through the ResponseUserInterface's *askForValue* method without using commas or periods, only with numeric values (Ex.: "R$ 10.50" must be sent as "1050").

messageDisplay
Description: Indicates the need to display a text message to the user.
Action: The message to be displayed is in the message parameter. A response to the SDK is not required.

showContactlessCard

Description: Indicates which stage of card reading the process is at, defined by returning the strings BEGIN, PROCESSING, DONE or ERROR.
Action: This callback can be used to indicate to the user the reading stage and the moment to move the card away from the NFC reader. A response to the SDK is not required.

askForDate
Description: Indicates the need to capture the date or time, the mask parameter identifies the type.
Action: A field for string capture must be provided and the captured string must be sent through the ResponseUserInterface's *askDate* method without the use of special characters, only with numeric values (Ex.: 10/11/2000 must be sent as "10112000").

cleanDisplay
Description: Indicates that the last displayed interface can be removed, usually due to processing progress.
Action: Clear the last displayed interface and display a progress indicator as some processing is in progress by the SDK. A response to the SDK is not required.

askForConfirm
Description: Indicates the need for user confirmation for an action.
Action: Display the received message and an interface for the user so they can confirm the action. The user selection must be sent in the ResponseUserInterface's *askForConfirm* method as a boolean.

askSelect
Description: Indicates the need for the user to select one of the items from the list received in the options parameter.
Action: It should display a list, where the user must choose one of the items provided in the options parameter array. This selection must be sent in the *askSelect* method of the ResponseUserInterface, the selected item must be sent in the selected parameter in the same way in which it was received in the input array.

showInsertCardMessage
Description: Indicates the start of card reading.
Action: It must display a screen for reading the card, the display parameter returns messages such as: "Approach the card" to help with user interaction. A response to the SDK is not required.

showPin
Description: Indicates the start of password reading, the displayed keyboard is under the full control of the SDK for application security. The display parameter presents an explanatory message to the user.
Action: You can display the message received in the message parameter. The password keyboard occupies around 60% of the bottom of the screen, avoid placing any essential UI elements in this area during this step. The same transaction value passed to the SDK API (paymentTransaction) MUST be displayed in the UI during this capture. Not following this guidance will result in non-compliance to the PCI MPoC. A response to the SDK is not required.

askForConfirmRefund
Description: Indicates the need to confirm cancellation transaction
Action: An object is passed to this function by the SDK (RefundConfirmData) with data provided during the cancellation transaction, such as date, NSU and value. This data should be displayed to the user and ask for a confirmation from the user. The confirmation must be sent to the ResponseUserInterface method askForConfirm with the Boolean parameter representing the user's choice.

showQRrCode
Description: Receives the Bitmap with the QR Code for the PIX Transaction
Action: A Bitmap is passed to this method with the QR code that is to be used for the PIX operation, it also receives a String in the pinCode parameter with the value for manual input.

hideQrCode
Description: Indicates the end of the transaction with QR Code
Action: After this method is called the operation has ended and you can expect the result as described in the Transaction Result chapter (11.5)

8.2 Connecting the App UI to the SDK

The object created from the ClientUserInterface interface must be connected to the SDK context, this is done through the connectUI method from the IAuttarSDK.

When this object is instantiated the following connection call must be made:

```
Auttar.getSDK().connectUI(_clientUserInterface)
```

This call must be made whenever the application wishes to instantiate a new object that will respond to communication callbacks.

8.3 Cancel Button

It is only possible to have the cancellation action on the card reading screen, with this specific cancel button configuration.

For that you need to create an xml named “top_read_card_activity.xml” containing a Button with the id “@+id/top_cancel_button”, this button does not actually need to be visibile, it only needs to be in the same screen position that the actual button of the card reading screen.

- Resource ids must not be different from top_read_card_activity.xml and @+id/top_cancel_button, it´s mandatory.
- Use of xml is mandatory; currently, we do not support inflated views or Android Compose.
- It´s only mandatory if you are going to implement a cancel button at the time of the card reading screen.

Example:

```
<?xml version="1.0" encoding="utf-8"?>
<androidx.constraintlayout.widget.ConstraintLayout xmlns:a
    xmlns:app="http://schemas.android.com/apk/res-auto"
    xmlns:tools="http://schemas.android.com/tools"
    android:layout_width="match_parent"
    android:layout_height="match_parent">

    <androidx.appcompat.widget.AppCompatButton
        android:id="@id/top_cancel_button"
        android:layout_width="0dp"
        android:layout_height="wrap_content"
        android:layout_margin="20dp"
        android:text="Cancelar"
        android:visibility="gone"
        app:layout_constraintBottom_toBottomOf="parent"
        app:layout_constraintEnd_toEndOf="parent"
        app:layout_constraintStart_toStartOf="parent"
        app:layout_constraintWidth_default="wrap"
        tools:visibility="visible" />

</androidx.constraintlayout.widget.ConstraintLayout>
```

8.4 Card Reading Screen Customization

The card reading screen (CardReadActivity) includes a security layer. If you need to modify elements of this screen, such as the status bar, you can override the theme applied to the activity and apply the desired customizations.

To do this, simply declare the activity in the AndroidManifest.xml file and override the corresponding theme.

It is important to highlight that when changing the theme, some additional configurations must be applied — such as setting a transparent background — among others, as shown in the example below:

Example:

```
<activity
    android:name="br.com.execucao.softtrsm.remote.view.CardReadActivity"
    android:theme="@style/CardReadTheme"
    tools:replace="android:theme" />
```

```
<style name="CardReadTheme" parent="AppTheme">
    <item name="android:windowFullscreen">true</item>
    <item name="android:windowIsTranslucent">true</item>
    <item name="android:windowEnterTransition">@null</item>
    <item name="android:windowExitTransition">@null</item>
    <item name="android:windowEnterAnimation">@null</item>
    <item name="android:windowExitAnimation">@null</item>
    <item name="android:windowBackground">@android:color/transparent</item>
    <item name="android:colorBackgroundCacheHint">@null</item>
    <item name="android:windowContentOverlay">@null</item>
    <item name="android:windowNoTitle">true</item>
    <item name="android:backgroundDimEnabled">false</item>
    <item name="android:windowAnimationStyle">@android:style/Animation</item>
</style>
```

9 Save Access Token

When transacting, it is necessary to send the access token. The token has a one-hour expiration window. If the token is still valid, the transaction proceeds. Otherwise, an error is returned.

In the case this token is not provided, an empty string or invalid token is provided, the transaction will fail, and an error code (S-15) will be returned on the Transaction Result.

Usage example:

```
Auttar.getSDK().setAccessToken(_accessToken)
```

Method signature:

```
/**
 * Sets the access token for the transaction.
 * The access token is used for authentication and is associated
  with the transaction for the TOP.
*/
fun setAccessToken(
    accessToken: String
)
```

10 Authentication

Once authentication with the Getnet APIs described in chapter 3.1 is complete, and the integration in chapter 6 is ready, it is now possible to carry out the authentication step. The SDK provides this through the *authenticationSilent* method found in the AuttarSDK object.

Method signature:

```
  /**
 * Performs Silent authentication.
 * For more details, refer to the ParameterLoginSilentInterface data
class.
 */
fun authenticationSilent(
    parameterLoginIntegration: SilentLoginParamsInterface,
    result: LoginResultListener
)
```

The ***authenticationSilent*** method takes two parameters:

- **parameterLoginIntegration (SilentLoginParamsInterface):** Object containing parameters necessary to carry out authentication, below we have the definition of the interface and which parameters must be provided. Remembering that the *bearerToken* and *appToken* are obtained by direct integration with our APIs.

```
interface SilentLoginParamsInterface {
    /**
     * Token generated in oauth/token, it returns an 'access-token'
     */
    val accessToken: String

    /**
     * App Token generated in top-auth-adp/app-token
     */
    val appToken: String

    /**
     * Document number (CNPJ)
     */
    val document: String

    /**
     * Package name of application
     */
    val packageName: String
```

```
    /**
     * Device's name
     */
    val deviceName: String

    /**
     * Nickname, optional
     */
    val nickName: String?

    /**
     * User's name logged in the App, optional
     */
    val user: String?
}
```

- **result (LoginResultListener):** callback for asynchronous processing, where the success or otherwise of execution will be returned through an object defined by the interface below:

```
  interface LoginResultInterface {
    /**
     * Result code, returns 0 when successful
     */
    val code: Int

    /**
     * Message about authentication, useful in case of failures
     */
    val message: String
}
```

After the successful return, the application will be ready to carry out transactions.

11 Transaction

To start a transaction, make sure that the chapters 7 and 8 were followed and the UI callbacks integration are in place.

The transaction flow starts with the method *paymentTransaction* accessible through the AuttarSDK object, receiving as parameters the transaction object, which may vary according to the type of transaction. Use transactionStateFlow to observer status of Transaction.

```
  /**
 * Executes a transaction using the new reactive API.
 *
 * This method no longer requires a listener. Use
[transactionStateFlow] to observe transaction states.
 */
fun paymentTransaction(
    data: DefaultData, listener: TransactionResultListener
)
```

```
  /**
 * Example to observer in Activity.
 *
 * [transactionStateFlow] to observe transaction states.
 */

lifecycleScope.launch {
  lifecycle.repeatOnLifecycle(Lifecycle.State.STARTED) {
    auttarSdk.transactionStateFlow.collect { transactionState ->
      if (transactionState is AuttarTransactionState.Done) {
        val tefResulTransaction = transactionState.result
      }
    }
  }
}
```

The data parameter is a DefaultData object that can be of the subtypes: CreditData, DebitData and CancellationData.

The transactionState.result when TransactionState.Done implementation that will inform the result of the transaction, more details on section 10.4.

After a transaction operation is started, the SDK will communicate the necessary steps, e.g.: request the user to hold the card near the Android device, through the UI callback implementation described in chapter 8.

If you wish to abort a transaction you can do it through the ResponseUserInterface, described in section 8.1.

11.1 Credit Transaction

Specific transaction for credit cards only, it allows for the use of installments through the card operator.

A CreditData object is passed to the data parameter of the *paymentTransaction* method to start this type of transaction.

```
  data class CreditData(
    /**
     * Transaction value in BRL (R$) without
     */
    val amount: BigDecimal,

    /**
     * Type of credit transaction, check the possible types in
TypeCreditTransaction
     */
    var type: TypeCreditTransaction,

    /**
     * Number of installments, only required for transaction with
   * installments
     */
    var numberInstallments: Int? = null,
}
```

11.2 Debit Transaction

Transaction used for debit cards only. This operation is started passing a DebitData object to the data parameter of the *paymentTransaction* method.

```
  data class DebitData(
    /**
     * Transaction value in BRL(R$)
     */
    val amount: BigDecimal,

    /**
     * Type of debit transaction, check the possible types in
TypeDebitTransaction
     */
    var type: TypeDebitTransaction,
}
```

11.3 Voucher Transaction

The SDK supports payment operations with vouchers, using the **VoucherData** class a parameter for the *paymentTransaction* method.

```
/**
 * Data Class used as a parameter for voucher payments
 */
data class VoucherData(
    /**
     * Transaction value in BRL (R$)
     */
    override val amount: BigDecimal,

    /**
     * Type of voucher operation
     */
    override var type: TypeVoucherTransaction,

    /**
     * Object for payment split configuration
     */
    override var splitPaymentIntegration: ISplitPaymentIntegration? = null
```

Voucher Operation Selection

To specify the type of voucher operation, use the **type** field of the **VoucherData** object, with the corresponding value from the **TypeVoucherTransaction** enum:

```
/**
 * Enum with options for voucher transactions
 * code is from OperationEnum from lib ctfclient
 */
enum class TypeVoucherTransaction(override val code: Int) :
AuttarTypeTransaction {
    DEB_VOUCHER(106) //OperationEnum.OP_DEB_VOUCHER.key
}
```

11.4 Cancellation

Used to cancel a previously successful debit or credit transaction with the CancellationData object.

For this operation the user will be prompted by the UI integration (chapter 7) to request the date of the original transaction. The value of the cancellation can be lower than the value of the original transaction, resulting in the refund of the inserted value.

```
  data class CancellationData(
    /**
     * Value of the transaction to be refunded in BRL (R$)
     */
    val amount: BigDecimal,

    /**
     * NSU of the original transaction, if not provided, the SDK will
request it during the transaction flow.
     * Note: The NSU can be found on the TefResult returned in the
transaction result callback, it is the number that identifies the
transaction in the CTF
     */
    var nsu: Int = 0,
}
```

11.5 Pix Operations

The SDK supports Pix transactions using QR Code or Wallet NFC. Pix operations are managed using the **PixData** class, which must be provided as a parameter to the *paymentTransaction* method.

```
/**
 * Data Class used for PIX transactions
 */
data class PixData(
    /**
     * Transaction value in BRL (R$)
     */
    override val amount: BigDecimal,

    /**
     * Type of PIX transaction
     */
    override var type: TypePixTransaction,
)
```

To specify which Pix transaction to execute, set the **type** property of the **PixData** object using the corresponding value from the **TypePixTransaction** enum:

```
enum class TypePixTransaction(val code: Int) {
    /**
     * Performs a PIX payment flow
     */
    PAGAMENTO_PIX(422), //OperationEnum.OP_PIX_PAYMENT.key

    /**
     * Returns the status of a PIX transaction
     */
    CONSULTA_PIX(423) // OperationEnum.OP_PIX_QUERY.key
```

```
    /**
     *  Performs a PIX refund payment flow
     */
    DEVOLUCAO_PIX(431) // OperationEnum.OP_PIX_RETURN.key
}
```

Pix Payment Flow

For Pix payments (**PAGAMENTO_PIX**):

- The SDK generates a QR Code and sends it via the **showQRCode** callback in **ClientUserInterface.**
- Simultaneously, the SDK activates the NFC antenna for Wallet approximation. The process follows standard card reading indications, using the **showContactlessCard** callback with ContactlessStatusEnum.
- Once payment is detected, the result is returned as described in the Transaction Result chapter (11.5).

Pix Query Flow

For Pix queries (**CONSULTA_PIX**):

- Additional parameters must be set in the **PixData** object:
    - **transactionId** and **receiverPSP**, or
    - **nsu** (when querying by NSU, also provide the transaction date).
- These fields allow the SDK to locate and return the status of a specific Pix transaction.

```
/**
 * ID of the previously executed Pix transaction
 * Note: Only used in Pix Query, see more in TypePixTransaction
 */
override var transactionId: String

/**
 * Receiver PSP of the previously executed Pix transaction
 * Note: Only used in Pix Query, see more in TypePixTransaction
 */
override var receiverPSP: String

/**
 * NSU of the previously executed Pix transaction
 * Note: Only used in Pix Query, see more in TypePixTransaction
 */
override var nsu: String

/**
 * Date of the previously executed Pix transaction
 * Note: Mandatory when querying by NSU. Must match the transaction date.
 */
override var transactionDate: Date = Date()
```

Pix Refund Flow

For Pix refunds (**DEVOLUCAO_PIX**):

- Use the same **PixData** structure, setting the **type** to **DEVOLUCAO_PIX**.
- Provide the necessary identifiers for the transaction to be refunded.

Important Recommendations

- Do not start a new operation while a Pix transaction is in progress.
- Disable the back button until the Pix operation result is returned to avoid interruptions

The response fields returned on TefResultInterface object indicate the status of the PIX transaction according to the table below.

returnCode	msgCTF	Description
00	PAGO	Completed
80	A_DEVOLVER	Pending refund
81	A_PAGAR	Pending payment
82	COMPLETAMENTE_DEVOLVIDO	Refund completed
83	PARCIALMENTE_DEVOLVIDO	Partially refunded
84	REMOVIDO_PELO_USUARIO_RECEBEDOR	Removed by the receiving user
85	REMOVIDO_PELO_PSP	Removed by PSP
86	EXPIRADO	Expired
87	INDEFINIDO	Undefined
88	NEGADO	Denied

11.6 Transaction Result

The transaction result is obtained through the callback (TransactionResultListener) sent at the start of the transaction together with the data necessary for it to occur.

The interface is defined as below:

```
  /**
 * Transaction Result Listener
 */
interface TransactionResultListener {
    /**
     * Callback for listening the transaction result
     * Invoked when the transaction is done
     */
    fun onResult(result: TefResultInterface)
}
```

The *onResult* method is invoked when the transaction is terminated for any reason, be it a success or failure. This method returns an object of type TefResultInterface containing information relevant to understanding the process.

In the list below we have the most important properties for understanding the result of the transaction, other properties can be consulted in the API documentation.

Property	Description
returnCode	Value 0 when the transaction was a success, otherwise check the codSupport field for a code specific to the problem encountered.
codSupport	Error code generated by any error or problem encountered during the transaction
nsuCTF	NSU transaction identifier code, used to cancel the operation.
flag	Card brand used for payment.
cardNumber	Card number masked (only 4 last digits visible).
dateTime	Transaction timestamp.

11.7 Transaction Hot Start

For optimizing the time necessary to start a transaction it is possible to give it a hot start to the SDK declaring the intention of starting a transaction through the method *initTransaction* from the AuttarSDK facade.

As an example, this method can be called when you start the capture of the transaction parameters from the user (value, type, installment), while the application prompt the user for that information, the SDK will be running start up process on the background.

Important to note that this is an ongoing optimization method, and results may vary version to version, and application to application.

11.8 Transaction Example

An example on how to start a credit transaction of value R$ 1000,00 with no installments:

```
//Access the SDK Facade
val auttarSdk = Auttar.getSDK()

//Optional hot start to the transaction process
auttarSdk.initTransaction()

//Capture transaction parameters with user interface
```

```
/* Application Implementation */

//Define the listener for the transaction result
val transactionPaymentResult = object : TransactionResultListener {
    override fun onResult(result: TefResult) {
        showResult(
            result.returnCode,
            result.display,
            result.messageAction,
            result.nsuCTF,
            result.customerPaymentReceipt
        )
    }
}
//Connect the application defined callback for interfaces
auttarSdk.connectUI(userInterface)
//Set the updated access token
auttarSdk.setAccessToken(ACCESS_TOKEN)
//Execute the transaction with the captured parameters
auttarSdk.paymentTransaction(
    CreditData(
        BigDecimal(1000),
        TypeCreditTransaction.CREDITO_GENERICO
    ),
    transactionPaymentResult
)
```

11.9 Transaction Error Return

The return transactional errors are accessible through the TefResultInterface, accessible through the *codSupport* property of the TefResult. Listed in the Appendix A at the end of this document, you can find some of the possible errors.

The error code is composed of a starting code followed by another code representing the error on the tables of this document. Examples being c20:2140 or a2005, always look for the last four digits when looking at the error tables.

12 Configurations

Some configuration data is accessible through the IUserConfiguration interface, accessible through the userConfiguration property of the IAuttarConfigSDK facade contained in the Auttar object.

```
val configSDK: IAuttarConfigSDK = Auttar.getConfigSDK()
```

The data available for consultation is related to the registered terminal, as in the example below we have a CNPJ consultation linked to previously carried out authentication:

```
val configSDK = Auttar.getConfigSDK()
val CNPJ = configSDK.userConfiguration.CNPJ.value
```

12.1 Submerchant

It is possible to configure the SDK to use a submerchant for the transactions processed by the SDK.

The submerchant data is set through the config facade shown above using the method:

```
/**
 * Set the data for sub merchant transactions.
 * @param subMerchant Data class with the sub merchant information.
 * @return Task result with code = 0 in case of success, otherwise
code = -1
 */
suspend fun setSubMerchant(subMerchant: SubMerchant) :
SubMerchantResult
```

The data class SubMerchant should hold all the information needed as described in the table below:

Parameter	Type	Max. Size	Details
id	String	15	Alphanumeric submerchant ID. Example: 1248ADC
city	String	13	Submerchant city, only letters allowed (a-z and A-Z). Example: SAO PAULO
state	String	2	Submerchant state, only letters allowed (a-z and A-Z) Example: SP
zipCode	String	8	Submerchant zip code, numbers only. Example: 90123456
cpfCnpj	String	11/14	Submerchant CPF or CNPJ, numbers only. Example: 12345678901234
address	String	40	Submerchant address, alphanumeric string with space and some special characters allowed (% $, . / & ( ) + = = < > - *) Example: AV NACOES UNIDAS, 1245
razaoSocial	String	99	Submerchant razao social, alphanumeric string with space and some special characters allowed (% $, . / & ( ) + = = < > - *)

			Exemplo: GETNET ADQUIRENCIA E SERVICOS PARA MEIOS DE PAGAMENTO S.A
nomeFantasia	String	99	Submerchant fantasy name, alphanumeric string with space and some special characters allowed (% $, . / & ( ) + = = < > - *) Example: Loja Santo Antônio da Pat
phone	String	11	Submerchant phone, numbers only. Example : 11987654321
url	String	255	Submerchant URL, mandatory only for e-commerce. Example: https://www.example.com.br OPTIONAL
idTransacao	String	40	Tag for partner internal control. Example: 47144560905_717e756e666d3e777833 OPTIONAL
idPagamento	String	20	Tag for partner internal control. Example: 000087806827933 OPTIONAL

After setting the submerchant the SubmerchantResult object will be returned with the operation result, in case of error the message field will contain a string describing the problem.

```
data class SubMerchantResult(
    var code: Int,
    var message: String? = null
)

enum class SubMerchantResultCodeEnum(val codeResult: Int) {
    INVALID_FIELD(-1),
    VALID(0)
}
```

IMPORTANT: The SDK will hold the last submerchant set and will automatically use it for every transaction executed. Setting a new one will replace the previous one.

You can check the submerchant information in use by the SDK through the config facade using the method below:

```
/**
 * Function to return the registered sub merchant
 * @return Returns the registered object SubMerchant or null if none
is registered.
```

```
 */
suspend fun getSubMerchant() : SubMerchant?
```

To remove the submerchant and use the default transaction method you can use the following method from the same facade:

```
/**
 *  Remove the stored sub merchant data
 */
suspend fun clearSubMerchant()
```

13 NFC Calibration

The Tap On Phone module provides functionality for detecting the position of the device's NFC antenna to facilitate the user experience. It is possible to implement a flow so that the user can recognize the best position to approach the card so that the reading can be done with greater precision.

This feature is demonstrated in the example project and its implementation takes place by applying the abstract class CalibrationTapOnPhoneFragment to the fragment with the desired interface.

In it we have implemented the callback and sequence methods for reading the card, where the current reading position is informed (*onReadingPosition*), if the position is effective, it will be returned in *onPositionRead*.

```
abstract class CalibrationTapOnPhoneFragment : Fragment() {

    internal val calibrationNfcFragment = CalibrationNfcFragment()

    /**
     * Initial state, not reading
     */
    abstract fun onIdLe()

    /**
     * Callback for errors
     */
    abstract fun onError(error: TopPositionError)

    /**
     * Reading the card at given position
     */
    abstract fun onReadingPosition(positionNFC: TopPositionNFC)

    /**
     * Finished reading
     */
    abstract fun onPositionRead(positionNFC: TopPositionNFC)

    /**
     * Indicates if there's a previous calibrated position
     */
    fun hasPrevious(): Boolean =
calibrationNfcFragment.nfcCalibration?.hasPrevious() ?: false

    /**
     * Indicates if there's a next calibration position
     */
    fun hasNext(): Boolean =
```

```
calibrationNfcFragment.nfcCalibration?.hasNext() ?: false

    /**
     * Set previously calibrated position
     */
    fun setPrevious() {
        calibrationNfcFragment.nfcCalibration?.setPrevious()
    }

    /**
     * Set upcoming calibrated position
     */
    fun setNext() {
        calibrationNfcFragment.nfcCalibration?.setNext()
    }

    /**
     * Restart calibration process.
     */
    fun restart() {
        calibrationNfcFragment.nfcCalibration?.restart()
    }

    /**
     * Pause calibration process
     */
    fun pause() {
        calibrationNfcFragment.nfcCalibration?.pause()
    }

    /**
     * Resume calibration process
     */
    fun resume() {
        calibrationNfcFragment.nfcCalibration?.resume()
    }
}
```

After closing the flow, the position will be stored locally by the SDK and can be accessed at the time of the transaction through the ITapConfig interface with the *getPositionReadConfigured* function as in the example below:

```
  import br.com.auttar.bc.top.extensions.getConfigTapOnPhone

  val tapConfig : ITapConfig = getConfigTapOnPhone().value
  val position : TopPositionNFC =
tapConfig.getPositionReadConfigured()
```

The position is defined by the TopPositionNFC enum:

```
enum class TopPositionNFC(val id: Int) {
    /**
     * Back center position
     */
    BACK_CENTER(PositionNFC.BACK_CENTER.id),

    /**
     * Back bottom position
     */
    BACK_BOTTOM(PositionNFC.BACK_BOTTOM.id),

    /**
     * Back top position
     */
    BACK_TOP(PositionNFC.BACK_TOP.id);
}
```

APPENDIX A

Blocking Code	Blocking Name	Description	Suggested Action
2000	GENERIC	Blocking was done by unmapped situation	Investigate logs, as this would not normally occur
2001	TAMPER_INSTALLATION	Blocking due to not being installed through Play Store	Reinstall the application through Play Store
2002	TAMPER_ROOT	Blocking due to device is rooted or has tools to root it	Revert the device root and uninstall any tools used to. If unsuccessful, replace the device
2003	TAMPER_MALWARE	Blocking due to a malicious app being installed in this device	Uninstall the malware application. The malware app package name is callbacked to the COTS application upon detection
2004	TAMPER_SETTINGS	Blocking due developer or other insecure options enabled	Disable the insecure setting. The detected setting is callbacked to the COTS application upon detection
2005	TAMPER_CAMERA	Blocking due to camera being used during card transaction	Instruct the user not to use the camera during the transaction. If the user didn't used, another malicious app may be trying to use it
2006	TAMPER_ATTESTATION	Blocking due to Safety Net attestation findings	Further investigation of the blocking cause must be made to check what the Safety Net attestation pinpointed. The specific error can be found on the error logs of the device. If the app failed attestation, reinstall through Play Store. If device failed attestation, replace the device
2007	TAMPER_HASH	Blocking due to hash sent by the app not matching the expected by server	Check if the hash mismatched was the signature hash or the app hash. If signature hash, reinstall app through Play Store. If app hash, unblock the device and ask the user to try again, as a Play Integrity attestation will be performed, and if no issues are found, a new app hash will be saved
2008	TAMPER_DOWNGRADE	Blocking due to SO, app, library or security patch downgrade	If the app was downgraded, reinstall the latest version through Play Store. If SO or security patch downgrade, revert this action by updating the device
2009	FRONTEND	Blocked through the front-end interface	Check with the user who blocked the device, as the user's email who blocked the device is listed with this blocking code
2010	ACQUIRER	Blocked through the acquirer backend	Check with the team who manages the blocking requests through API
2011	TAMPER_DEXGUARD	Blocked due to DexGuard verifications	The specific security problem Dexguard detected is callbacked to the COTS application and should be displayed so the user can see the problem and revert it. If unsuccessful, replace the device

2012	TAMPER_PLAY_INTEGRITY	Blocked due to Play Integrity API findings	Further investigation of the blocking cause must be made to check what the Play Integrity attestation pinpointed. The specific error can be found on the error logs of the device. If the app failed attestation, reinstall through Play Store. If device failed attestation, replace the device
2013	TAMPER_HARDWARE	Blocked due to hardware verifications finding different hardware than reported before	Replace the device, as device hardware should never change
2014	BLOCKED_BLACKLIST	Blocked due this device being in the blacklist (software or hardware)	Device is blacklisted, replace the device
2015	TAMPER_APPLICATION_CHECKSUM	Blocked due to application checksum validation	Reinstall app through Play Store
2016	TAMPER_SENSORS_FAILED	Blocked due to sensors used for entropy not behaving correctly	Replace the device
2017	BLOCKED_LOGIC_NUMBER	This logic number has been blocked due to being used by another device	Check if the device is trying to communicate with a new logic number. If so, the device should be unblocked, otherwise it must remain blocked.

Table 4. Blocking codes table

Result Code	Result Name	Description
0	OK	Successful
1000	GENERIC_ERROR	Generic error during operation
1001	TERMINAL_BLOCKED	Terminal is blocked
1002	ERROR_REQUEST_CRYPTOGRAPHY	Unable to open request cryptogram
1003	ERROR_TERMINAL_VALIDATIONS	Device failed hardware or software validations
1004	ERROR_TERMINAL_CLOSED	Device has closed the socket (the device will never receive this error)
1005	ERROR_SERVER_ERROR	Server found a fatal error and probably sent a http 500 status code (the device will never receive this error)
1010	ERROR_JSON	Error reading JSON
1011	ERROR_JSON_MISSING_FIELD	Missing field on request JSON
1012	ERROR_JSON_CNPJ	CNPJ not found
1013	ERROR_JSON_TERMINAL	Serial not found
1014	ERROR_JSON_APP	App not found
1015	ERROR_JSON_VERSION	App version not found

1020	ERROR_DATE_COMM	Error processing date/time of the communication
1021	ERROR_DATE_COMM_EXCEEDS_LIMIT	Sent date/time is too far off from the server date/time
1022	ERROR_DATE_COMM_TOO_FAST	Too many communication attempts in a short time
1024	ERROR_TRANSACTION_LIMIT_EXCEEDED	Transaction limit was exceeded
1030	ERROR_HASH	Error during origin validation
1031	ERROR_HASH_NO_MASTER	Master .APK not present on APK list
1032	ERROR_HASH_NOT_FOUND	An APK name sent was not found on the server
1033	ERROR_HASH_NOT_MATCH	Hash does not match
1034	ERROR_HASH_CERTIFICATE_NOT_MATCH	Certificate hash does not match
1040	ERROR_VERSION	Error during the version validation
1041	ERROR_VERSION_DOWNGRADE	The app version has been downgraded
1042	ERROR_VERSION_EXPIRED	This app version is expired
1050	ERROR_ATTESTATION	Error during the device attestation. If returned during another request other than an attestation, it indicates that the previously attestation has expired, and the gateway is demanding a new attestation
1051	ERROR_ATTESTATION_JWT	Error on JWT field
1052	ERROR_ATTESTATION_CERTIFICATE	Error on the certificate
1053	ERROR_ATTESTATION_CTS	CtsProfileMatch result is false
1054	ERROR_ATTESTATION_DATE	Certificate date/time is too far off from the server date/time
1055	ERROR_ATTESTATION_NONCE	Nonce from the attestation result does not match
1056	ERROR_ATTESTATION_PACKAGE_NAME	Package name from the attestation result does not match
1057	ERROR_ATTESTATION_SIGNATURE	Invalid signature of the certificate
1058	ERROR_ATTESTATION_SIGNATURE_CHECK	Unable to check the certificate signature
1059	ERROR_ATTESTATION_RNG	RBG algorithm validation failed
1060	ERROR_NO_KEYS	Error during the generation of the keys
1061	ERROR_NO_KEYS_CNPJ	No keys available for this acquirer
1062	ERROR_NO_KEYS_MAP	Unable to retrieve keys for this app version
1063	ERROR_NO_KEYS_HSM	HSM was unable to provide keys
1064	ERROR_NO_ENTROPY_HSM	HSM was unable to provide entropy
1070	ERROR_INTEGRITY	Error during Integrity validation
1071	ERROR_INTEGRITY_LICENSE	Integrity check of app license failed
1072	ERROR_INTEGRITY_APP	Integrity check of app version failed
1073	ERROR_INTEGRITY_DEVICE	Integrity check of device failed

Table 5. Result codes table

Result Code	Result Name	Description
0	SUCCESS	Successful

1	PARAMETER_INVALID_USER	Inserted user is invalid
2	PARAMETER_INVALID_PASSWORD	Insert password is is invalid
3	PARAMETER_INVALID_CNPJ	Inserted CPNJ is invalid
4	PARAMETER_INVALID_STORECODE	Inserted storecode is invalid
5	PARAMETER_EMPTY_GROUP_EC	Inserted groupEC is invalid
6	PARAMETER_EMPTY_CODE_ACTIVE	Inserted codeActive is invalid
7	HTTP_ERROR	HTTP error on login request
8	ERROR_CERTIFICATE_MD5_NOT_CHECK	Fail to check MD5 certificate
9	ERROR_CERTIFICATE_GET_MD5	Fail to access MD5 certificate
10	PARAMETER_CONTEXT_NULL	Android Context is null
11	ERROR_WRITE_CERTIFICATE	Fail to write certificate
12	ERROR_GET_ANDROID_ID	Fail to get android ID
13	ERROR_SDK_RUNNING_STANDALONE	SDK selected environment is standalone

Table 6. LoginResult codes table

Result Code	Result Name	Description
0	SUCCESS	Successful
9	ATT_KEYS	Fail to get attestation keys
10	ATT_INTEGRITY	Fail to get attestation integrity
11	ATT_INFO	Fail to get attestation info
12	PP_TIMEOUT	Operation time limit
13	PP_CANCEL	Operation canceled by user
34	PP_COMMTOUT	Timeout waiting for pinpad
53	ERROR_AUTHENTICITY_FAILED	Fail to get auth token for app host
70	PP_CARDAPPNAV	Card with no application available
79	PP_CARDBLOCKED	Card is blocked
80	PP_CTLSSMULTIPLE	Simultaneous multiple cards at CTLS reader
81	PP_CTLSSCOMMERR	Communication error on pinpad and contactless
82	PP_CTLSSINVALIDAT	CTLS invalid
83	PP_CTLSSPROBLEMS	CTLS Reading error
84	PP_CTLSSAPPNAV	CTLS with no application available
85	PP_CTLSSAPPNAUT	Card not supported
87	ST_CTLSIFCHG	CTLS not supported
-2147483648	PP_DEVICE_NOT_SAFE	Device is not safe

Table 7. Attestation codes table

Result Code	Result Name	Description
5300	VALUE_NOT_FOUNDED	Error, transaction amount is required
5301	CARD_NOT_VALID	Invalid card
5302	CARD_OUTDATED	Outdated card
5303	VALIDATE_DATE_INVALID	Validade date invalid, check the card
5304	SECURITY_CODE_INVALID	Security code invalid, check the card
5305	SERVICE_TAX_OVER	Service tax over
5306	OPERATION_UNAUTHORIZED	Operation unauthorized
5307	INVALID_DATA	Invalid data, check the data of transaction
5308	MINIMUM_VALUE_INVALID	Minimum value invalid, try again with another installment
5309	INSTALLMENTS_NUMBERS_INVALID	Installments numbers invalid, try again with another installment
5310	INSTALLMENTS_NUMBER_OVER	Installments numbers over, try again with another installment
5311	INITIAL_VALUE_OVER	Intitial value over
5312	INITIAL_VALUE_INVALID	Intitial value invalid
5313	INVALID_DATE	Invalid date
5314	TERM_OVERS_LIMIT	Term overs limit, check the term
5316	NSU_INVALID	Invalid NSU
5317	OPERATION_CANCELED_BY_USER	Operation canceled by user
5318	INVALID_DOCUMENT	Invalid document
5319	VALUE_DOCUMENT	Invalid document
5327	PINPAD_CONNECTION_FAILURE	Pinpad failure in connection
5329	READING_FAILURE	Error on read the card
5364	NUMBER_OF_INSTALLMENTS_OVER_LIMIT	Number of installments over limit
5366	TRANSACTION_INCOMPATIBLE	Invalid installment type
5412	FALLBACK_DISABLED	Error, fallback disabled for Tap On Phone Any time a card reading fails this error will be returned
55	INVALID_PASSWORD	Error, invalid password
0ES	UNAUTHORIZED	Transaction not authorized by Authorizer
F5	TRANSACTION_DENIED_BY_CARD	Transaction denied by card provider
E2	REFUND_CARD_ERROR	Error, refund details unavailable, sale not found

Table 8. Client error codes table

Result Code	Result Name	Description
-1	INPUT_TRANSACTION	Error processing transaction parameters
-2	PROCESS_CLIENT	Error processing transaction parameters in client
-3	DB_TRANSACTION_NO_DATA	Failed to find transaction

-4	DB_TRANSACTION_NOT_PENDING	Failed, the transaction is not pending
-5	DB_TEF_RESULT_NO_DATA	Failed, no data in last transaction
-6	MULTI_TRANSACTION_RUNNING	Failed, multi transaction is running
-7	MULTI_TRANSACTION_NO_DATA	Failed, no data for multi transaction
-8	STANDALONE_NOT_CONFIGURED	Error, when using standalone it is necessary to configure the terminal and communication
-9	HOMOLOG_PROD_NOT_CONFIGURED	Error, authentication required
-10	NFC_COMPATIBILITY_ERROR	Equipment not compatible with tap on phone
-11	NFC_DISABLE	Error, NFC disabled, enable and restart transaction
-12	TOP_DND_UNAUTHORIZED	Error, need to enable do not disturb
-14	PIN_PAD_BLUETOOTH_DISABLED	Error, need to enable bluetooth
-15	TOKEN_EXPIRED_ERROR	Your login token has expired or is about to expire.
-16	LOCATION_PERMISSION_DENIED	Error, location permission DENIED, GRANTED and restart transaction
-17	LOCATION_DISABLE	Error, location permission DENIED, GRANTED and restart transaction

Table 9. Transaction error codes table

Result Code	Result Name	Description
0241	TERMINAL_NOT_ACTIVATED	Terminal is not activated
0202	COMMUNICATION_ERROR2	Error on communication, check the internet connection
0203	COMMUNICATION_ERROR3	Error on communication, check the internet connection
0210	TIMEOUT	Error on communication, time out
0010	INTERNET_ERROR	Error on communication, check the internet connection

Table 10. Connection error codes table

Result Code	Result Name	Description
-1	INPUT_TRANSACTION	Transaction parameter error
-2	PROCESS_CLIENT	Unexpected error during payment flow
-3	DB_TRANSACTION_NO_DATA	Attempt to finalize a transaction when there are no records for the day
-4	DB_TRANSACTION_NOT_PENDING	Attempt to finalize a transaction that is not pending
-5	DB_TEF_RESULT_NO_DATA	Failed to find Tef Result for finalizing Multi Transaction
-6	MULTI_TRANSACTION_RUNNING	Multi Transaction in progress
-7	MULTI_TRANSACTION_NO_DATA	Error finalizing Multi Transaction
-8	STANDALONE_NOT_CONFIGURED	SDK initialization performed incorrectly
-9	HOMOLOG_PROD_NOT_CONFIGURED	SDK initialization performed incorrectly
-10	NFC_COMPATIBILITY_ERROR	NFC functionality not recognized on the device

-11	NFC_DISABLE	NFC functionality disabled
-12	TOP_DND_UNAUTHORIZED	Android Do Not Disturb permission not granted
-14	PIN_PAD_BLUETOOTH_DISABLED	Bluetooth Pinpad disabled
-15	TOKEN_EXPIRED_ERROR	App Token Expired
-16	LOCATION_PERMISSION_DENIED	Location permissions not granted
-17	LOCATION_DISABLE	Location disabled on the Android device
-18	DB_TEF_RESULT_NSU_NOT_FOUND	Transaction record not found for the provided NSU
-61	KEYSTORE_NOT_VALID	Failed to create/use the local Keystore for attestation
502	BAD_GATEWAY	A&M servers returned 502 for Attestation, Info, and Key operations
-2024	CLIENT_ERROR_ON_TOP	Internal use
3000	CANCEL_USER_PASSWORD	Transaction canceled by the user during PIN entry
3001	CANCEL_USER_BACKGROUND	Transaction canceled by the user during the transaction
2005	TAMPER_CAMERA	Device camera usage detected during the transaction
2016	TAMPER_SENSORS_FAILED	Failure in using sensors to generate random keys for the transaction
2004	TAMPER_SETTINGS	Sensitive Android settings detected during the transaction

Table 11. TOP error codes table

Change Log

Date	Author	Version	Comments
2023-01-30	Michel Martinez	1.0	Document creation.
2023-07-29	Lucas Santos	1.1	Adding information about calibration feature
2024-04-19	Lucas Santos	1.2	Reformulating document to add more general guidance and security requirements.
2024-04-22	Lucas Santos	1.2a	Document translation
2024-04-22	Lucas Santos	1.3	Adding Initialization chapter and extraction rulers
2024-05-10	Lucas Santos	1.4	Adding data management
2024-05-21	Lucas Santos	1.4a	Adding clarifications for MpoC on value requirement and security update.
2024-05-23	Lucas Santos	1.4b/c	Clarification on MpoC compliance
2024-05-27	Lino Veloso	1.4c	Include token to transaction
2024-06-05	Lucas Santos	1.4d	Clarification on communication of vulnerabilities
2024-06-06	Lucas Santos	1.5	Adding new sensitive assets table
2024-06-14	Lucas Santos	1.5a	Removing DrawOverlay permission from the list since is no longer required.
2024-06-14	Lucas Santos	1.5b	Updated the Android version requirement
2024-06-19	Lucas Santos	1.5c	Added table for the tamper behavior
2024-07-03	Lino Veloso	1.6	Added new Params for device control
2024-07-08	Lino Veloso	1.6a	Now SDK also accepts document inside Silent Login Params already encrypted in Base64
2024-07-19	Lucas Santos	1.7	Adjusting new permissions
2024-07-26	Lucas Santos	1.8	Updating according to MPoC requirements: clarifying token implementation, class extensions and code examples.
2024-08-06	Jackson Oliveira	1.9	Adding all possible codes to return transaction errors
2024-09-11	Victor Oliveira	1.10	Adjust integration with new version sdk 4.4.0
2024-12-19	Lucas Santos	1.11	Grammar review, adding new function for hotstart
2024-12-31	Eduardo Torres	1.12	Added more error return tables
2025-01-24	Lucas Santos	1.12a	Version review
2025-01-28	Lucas Santos	1.13	Added submerchant details
2025-04-28	Lucas Santos	1.14	Added Pix Operations
2025-06-24	Lucas Santos	1.15	Adding SDK Errors table
2025-06-24	Victor Oliveira	1.16	Change example transaction to use observerFlow
2025-06-22	Jackson Oliveira	1.16a	Added transaction date in PixOperation

2025-12-15	Eduardo Torres	1.17	Added Pix Devolution
2026-03-18	Jackson Oliveira	1.18	Added Voucher Transaction